

# Relatório de Evolução — Sprint 03

Projeto ChargeGrid Intelligence — EV Challenge GoodWe (FIAP, Grupo 7)

## 1. Resumo da evolução (Sprints 1/2 → Sprint 03)

Nas Sprints 1 e 2, o chatbot GoodWe/ChargeGrid era implementado como uma sequência manual de chamadas à API do LLM: uma função Python concatenava o system prompt, o histórico (mantido em uma lista comum) e a pergunta do usuário a cada turno, sem orquestração formal, sem guardrails testados sistematicamente e sem comparação objetiva entre modelos. Na Sprint 03, o núcleo conversacional foi refatorado para usar o framework de agentes **LangGraph**: o fluxo passou a ser modelado como um grafo de estados (*StateGraph*), a memória por sessão passou a ser gerenciada por um checkpointer nativo (*MemorySaver*), e uma suíte automatizada de testes funcionais, de memória e de segurança passou a ser executada contra múltiplos provedores de LLM (OpenAI e Google Gemini), com resultados exportados para `relatorio_modelos.md`.

## 2. Refatoração — decisões técnicas e trade-offs

**Framework escolhido:** LangGraph, por permitir modelar o agente como grafo de estados, oferecer checkpointer nativo para memória por sessão e ser agnóstico ao provedor de LLM (bastando trocar o objeto *ChatModel* injetado no nó do agente). Essa escolha viabilizou diretamente o requisito de comparação entre modelos, já que a mesma orquestração roda tanto sobre ChatOpenAI quanto sobre ChatGoogleGenerativeAI.

**Principais trade-offs:** (1) curva de aprendizado maior do que a versão manual das Sprints 1/2; (2) o checkpointer padrão (*MemorySaver*) mantém a memória apenas em RAM — suficiente para a demonstração da sprint, mas não persistente entre reinícios do kernel; (3) depender de uma biblioteca de terceiros introduz risco de *breaking changes* entre versões, mitigado fixando as versões no `pip install`.

## 3. Comparativo antes × depois

| Aspecto                         | Sprints 1/2 (manual)                                     | Sprint 03 (LangGraph)                                                                      |
|---------------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Arquitetura                     | Chamada manual e sequencial à API (função Python)        | Script de agente LangGraph ( <i>StateGraph</i> + checkpointer), reutilizável entre sessões |
| Memória                         | Lista Python em memória local, montada manualmente       | Gerenciada pelo <i>MemorySaver</i> (thread_id, testada e validada)                         |
| Seleção de modelo               | Escolha ad hoc (GPT-4o-mini), sem suíte comparativa      | Baseada em suíte automatizada — modelo escolhido: gemini-3.1-flash-lite                    |
| Testes de segurança             | Não formalizados (nenhum caso de Prompt Injection)       | Focalizados e automatizados (Prompt Injection, escopo, jurídico, financeiro, elétrico)     |
| Medição de latência/tokens      | Qualitativa apenas ('Gemini mais rápido ~1-2s, GPT ~2s') | Automática por turno — latência média mínima observada: 0.94 s (gpt-4o-mini)               |
| Nº de casos de teste funcionais | 5 (avaliação manual: Adequada/Parcialmente/Inadequada)   | 40 (avaliação automática por palavra-chave: OK/REVISAR)                                    |

Resultados quantitativos da suíte automatizada (execução real, Seção 7 do notebook):

| Modelo/config.                | Latência média (s) | Tokens médios (aprox.) | Taxa de sucesso |
|-------------------------------|--------------------|------------------------|-----------------|
| gemini-3.1-flash-lite (T=0.3) | 19.13              | 173.67                 | 75.0%           |
| gpt-4o-mini (T=0.3)           | 0.94               | 54.33                  | 75.0%           |
| gpt-4o-mini (T=0.9)           | 1.17               | 64.67                  | 75.0%           |

**Modelo escolhido para a versão final: gemini-3.1-flash-lite (T=0.3)**, com base na maior taxa de sucesso combinada (testes funcionais + memória + segurança) da suíte automatizada — decisão orientada por dados, e não por preferência do grupo. Detalhamento completo em `relatorio_modelos.md`.

## 4. Problemas encontrados e soluções

### Problema 1 — Memória perdida entre execuções de célula

**Alternativas consideradas:** Alternativas: (a) variável global manual; (b) checkpointer persistente em disco/banco; (c) MemorySaver do LangGraph em RAM.

**Solução adotada:** Adotado (c): MemorySaver indexado por thread\_id.

**Justificativa:** Atende ao requisito de memória por sessão com baixa complexidade; persistência em disco fica fora do escopo de uma demonstração local.

### Problema 2 — Avaliar guardrails de forma objetiva

**Alternativas consideradas:** Alternativas: (a) leitura manual de cada resposta; (b) um segundo LLM como juiz; (c) checagem determinística por palavras-chave.

**Solução adotada:** Adotado (c): heurística por palavras-chave, com revisão manual dos casos sinalizados.

**Justificativa:** Reprodutível e sem custo/latência extra de um LLM avaliador; suficiente para o volume de testes da sprint.

### Problema 3 — Comparar modelos de provedores diferentes de forma justa

**Alternativas consideradas:** Alternativas: (a) comparação subjetiva 'na sensação'; (b) mesma suíte e mesmos parâmetros para todos os provedores, variando só o modelo.

**Solução adotada:** Adotado (b): suíte e parâmetros fixos, com experimento isolado de temperature.

**Justificativa:** Garante que diferenças observadas sejam atribuíveis ao modelo, não a perguntas/parâmetros distintos, tornando a escolha final defensável com dados.

## 5. Divisão da equipe

| Nome                           | RM     | Principal responsabilidade                                          |
|--------------------------------|--------|---------------------------------------------------------------------|
| Renan Fracalossi Mano da Silva | 569610 | Engenharia de Agentes (Dev Core) — arquitetura LangGraph e memória  |
| Gabriel Barbosa Furin          | 572941 | Guardrails e testes de segurança (Prompt Injection e escopo)        |
| Gabriel de Almeida Santos      | 569395 | Comparação entre modelos — harness de testes e relatorio_modelos.md |
| Herbert Soares de Jesus        | 571507 | Interface de chat e integração da base de conhecimento              |
| Lucas Kiodi Moraca             | 571004 | Documentação, comparativo antes x depois e relatório de evolução    |

Relatório gerado automaticamente a partir dos resultados reais da execução da Seção 7 deste notebook.